



August 2026

Prepared for
Origin ARM

Audited by
Panda

yAudit Origin ARM Pull Request Review

Smart Contract Security Assessment

Contents

1	Review Summary	2
1.1	Protocol Overview	2
1.2	Audit Scope	2
1.3	Risk Assessment Framework	2
1.3.1	Severity Classification	2
1.4	Key Findings	3
1.5	Overall Assessment	3
2	Audit Overview	3
2.1	Project Information	3
2.2	Audit Timeline	3
2.3	Audit Resources	4
2.4	Critical Findings	4
2.5	High Findings	4
2.6	Medium Findings	4
2.7	Low Findings	4
2.7.1	Deposits after catastrophic losses can capture recoveries owed to pre-loss LPs	4
2.8	Gas Savings Findings	6
2.8.1	Ether.fi adapters can use transient storage during claims	6
2.9	Informational Findings	6

1 Review Summary

1.1 Protocol Overview

Origin's Automated Redemption Manager (ARM) contracts provide liquidity for protocol redemption flows and expose an ERC-4626-like interface for liquidity providers. The reviewed MultiAssetARM logic accounts for deposits, LP shares, fees, and asynchronous withdrawals, while the Ether.fi asset adapters convert eETH and weETH withdrawal requests into WETH for the ARM.

1.2 Audit Scope

This review covers the three implementation contracts modified by OriginProtocol/arm-oeth pull requests #311 and #320,

```
src/contracts/AbstractARM.sol
src/contracts/adapters/EtherFiAssetAdapter.sol
src/contracts/adapters/WeETHAssetAdapter.sol
```

1.3 Risk Assessment Framework

1.3.1 Severity Classification

Severity	Description	Potential Impact
Critical	Immediate threat to user funds or protocol integrity	Direct loss of funds, protocol compromise
High	Significant security risk requiring urgent attention	Potential fund loss, major functionality disruption
Medium	Important issue that should be addressed	Limited fund risk, functionality concerns
Low	Minor issue with minimal impact	Best practice violations, minor inefficiencies
Undetermined	Findings whose impact could not be fully assessed within the time constraints of the engagement. These issues may range from low to critical severity, and although their exact consequences remain uncertain, they present a sufficient potential risk to warrant attention and remediation.	Varies based on actual severity
Gas	Findings that can improve the gas efficiency of the contracts.	Increased transaction costs
Informational	Code quality and best practice recommendations	Reduced maintainability and readability

Table 1: severity classification

1.4 Key Findings

Breakdown of Finding Impacts

Impact Level	Count
■ Critical	0
■ High	0
■ Medium	0
■ Low	1
■ Informational	0

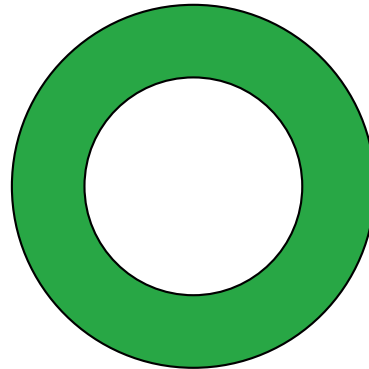


Figure 1: Distribution of security findings by impact level

1.5 Overall Assessment

The review identified one low-severity issue in PR #311: its fixed asset-floor guard stops deposits only at or below the floor, so one additional atomic unit permits a depositor to mint nearly all shares after a catastrophic loss and capture a later recovery. No actionable issue was identified in PR #320; its receive gate correctly causes unauthorized Ether.fi claims to revert while preserving adapter-initiated claims. PR #311 should not be treated as a complete remediation until recovery protection is made persistent and independent of the ARM's instantaneous token balance.

2 Audit Overview

2.1 Project Information

Protocol Name: Origin ARM

Repository: <https://github.com/OriginProtocol/arm-oeth>

Commit Hashes:

- 77eba86d9ce28bcbb8952b0315df4d851ea79071
- 1f048a831afcb4bfa68c2a9c879eec9bb517bbf2

Commit URLs:

- <https://github.com/OriginProtocol/arm-oeth/commit/77eba86d9ce28bcbb8952b0315df4d851ea79071>
- <https://github.com/OriginProtocol/arm-oeth/commit/1f048a831afcb4bfa68c2a9c879eec9bb517bbf2>

2.2 Audit Timeline

The audit was conducted on August 5, 2026.

2.3 Audit Resources

- Origin ARM pull request #311
- Origin ARM pull request #320

2.4 Critical Findings

None.

2.5 High Findings

None.

2.6 Medium Findings

None.

2.7 Low Findings

2.7.1 Deposits after catastrophic losses can capture recoveries owed to pre-loss LPs

PR #311 attempts to stop a new depositor from acquiring most of the ARM share supply after a catastrophic loss and capturing a later recovery belonging to existing LPs. However, the new guard applies only while assets are at or below the fixed `MIN_LIQUIDITY` floor. Moving assets one atomic unit above that floor enables the same deposit and recovery capture that the PR intends to prevent. PR #311 attempts to stop a new depositor from acquiring most of the ARM share supply after a catastrophic loss and capturing a later recovery belonging to existing LPs. However, the new guard applies only while assets are at or below the fixed `MIN_LIQUIDITY` floor. Moving assets one atomic unit above that floor enables the same deposit and recovery capture that the PR intends to prevent.

Technical Details

PR #311 adds the following deposit restriction:

```
1 uint256 grossAssets = _availableAssets();
2 uint256 feesAccruedMem = feesAccrued;

4 bool atAssetFloor = feesAccruedMem + MIN_LIQUIDITY >= grossAssets;
5 bool hasLiveLps = totalSupply() > MIN_TOTAL_SUPPLY;
6 if (atAssetFloor && (hasLiveLps || feesAccruedMem != 0 || reservedWithdrawLiquidity !=
  0)) revert Insolvent();
```

The restriction stops applying as soon as:

```
1 grossAssets > feesAccrued + MIN_LIQUIDITY
```

This threshold is unrelated to the size of the loss or the live share supply. Consequently, deposits are allowed when assets are only one atomic unit above the floor, even though the assets-per-share ratio can remain catastrophically low.

For example, consider an 18-decimal ARM:

1. Alice deposits 100 WETH and receives approximately 100 ARM shares.
2. A loss leaves the ARM with only `MIN_LIQUIDITY == 1e12` wei while Alice's shares remain outstanding.
3. Bob's deposit reverts because the ARM is exactly at the floor.
4. Bob transfers 1 wei of WETH directly to the ARM, increasing `grossAssets` to `MIN_LIQUIDITY + 1`.
5. The new guard no longer applies, so Bob deposits 100 WETH.
6. The existing mint formula gives Bob approximately `10,000,000,099.99` ARM shares, or approximately `99.999999%` of the post-deposit supply.
7. Bob consequently captures approximately `99.999999` WETH of a later 100 WETH recovery.

For a six-decimal liquidity asset, the guard can similarly be crossed with one atomic unit, or `0.000001` tokens.

The same outcome occurs without a direct donation whenever the loss naturally leaves assets slightly above `MIN_LIQUIDITY`. Therefore, PR #311 blocks only one narrow balance state and does not prevent the recovery-capture scenario described in its security rationale.

The behavior is demonstrated by `test_Deposit_AssetFloorGuardBypassedByOneUnitDonation`, which passes for both the 6- and 18-decimal ARM configurations.

Impact

Low. PR #311 does not prevent an authorized depositor from acquiring nearly the entire share supply after a catastrophic loss and capturing nearly all of a subsequent recovery. The depositor only needs the ARM's assets to exceed the fixed floor by one atomic unit and must have sufficient deposit authorization and cap headroom.

Recommendations

- Allow deposits up to a percentage of loss or block deposits when the vault incurs losses.
- In case of an attack where one would have got most of the shares, the funds need to be recovered with a smart contract upgrade and could be distributed via a merkle distributor.

Developer Response

I disagree that a post-loss recovery necessarily belongs exclusively to pre-loss LPs. ARM shares are priced using the current NAV. Existing LPs bear recognized losses, while new LPs entering at the impaired NAV receive shares at that prevailing price. From that point onward, all LP shares participate equally in future gains and losses.

2.8 Gas Savings Findings

2.8.1 Ether.fi adapters can use transient storage during claims

Technical Details

`EtherFiAssetAdapter.redeem` and `WeETHAssetAdapter.redeem` set `claimingEtherFi` before calling `batchClaimWithdraw` and clear it afterward. Their `receive` functions read this variable to ensure that ETH is accepted only during an adapter-initiated claim.

Since this value is required only for the duration of the transaction, persistent storage introduces unnecessary gas costs during redemptions.

Impact

Gas savings.

Recommendation

Use transient storage for `claimingEtherFi`. This can be implemented by upgrading to Solidity 0.8.28 or later, targeting the Cancun EVM version, and declaring the variable with the native `transient` keyword.

Developer Response

Changes implemented in [e6aa2011](#).

2.9 Informational Findings

None.



yAudit Origin ARM Pull Request Review

Completed 2026-08-05